AMAZON BRAZIL SQL ANALYSIS

Background

Amazon, a global leader in e-commerce, has achieved significant success in markets like the U.S., Europe, and Asia. In Brazil, Amazon connects small and medium businesses with millions of customers, becoming a key player. Given the similarities between Brazil and India, such as large populations and diverse consumer bases, there's an opportunity to replicate success in India.

ANALYSIS – I

Q.1	**To simplify its financial reports, Amazon India needs to standardize payment values.** Round the average payment values to integer (no decimal) for each payment type and display the results sorted in ascending order. ○ **Output**: payment_type, rounded_avg_payment

Query | Query History

```
SELECT
    payment_type,
    ROUND(AVG(payment_value)) AS rounded_avg_payment
FROM amazon_brazil."Payments"
GROUP BY payment_type
ORDER BY rounded_avg_payment ASC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

	payment_type character varying	rounded_avg_payment double precision
1	not_defined	0
2	voucher	66
3	debit_card	143
4	boleto	145
5	credit_card	163

Q.2	**To refine its payment strategy, Amazon India wants to know the distribution of orders by payment type.** Calculate the percentage of total orders for each payment type, rounded to one decimal place, and display them in descending order ○ **Output**: payment_type, percentage_orders

Query | Query History

```sql
-- 2. % of orders by payment type

SELECT
    payment_type,
    ROUND(COUNT(DISTINCT order_id) * 100.0 /
          (SELECT COUNT(DISTINCT order_id) FROM amazon_brazil.payments), 1)
          AS percentage_orders
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY percentage_orders DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

	payment_type character varying	percentage_orders numeric
1	credit_card	76.9
2	boleto	19.9
3	voucher	3.9
4	debit_card	1.5
5	not_defined	0.0

Q.3	**Amazon India seeks to create targeted promotions for products within specific price ranges.** Identify all products priced between 100 and 500 BRL that contain the word 'Smart' in their name. Display these products, sorted by price in descending order. **Output**: product_id, price

Query | Query History

```sql
SELECT
    product_id,
    price
FROM amazon_brazil."order_items"
WHERE price BETWEEN 100 AND 500
  AND product_id IN (SELECT product_id
      FROM amazon_brazil."product"
      WHERE product_category_name LIKE '%Smart%')
ORDER BY price DESC;


SELECT
    oi.product_id,
    oi.price
FROM amazon_brazil."order_items" oi
INNER JOIN amazon_brazil."product" p
    ON oi.product_id = p.product_id
WHERE oi.price BETWEEN 100 AND 500 AND p.product_category_name LIKE '%smart%'
ORDER BY oi.price DESC;
```

Data Output | Messages | Notifications

	product_id character varying	price double precision

Q.4	**To identify seasonal sales patterns, Amazon India needs to focus on the most successful months.** Determine the top 3 months with the highest total sales value, rounded to the nearest integer. ○ **Output**: month, total_sales
Top 3 months with highest total sales (rounded)	

Query Query History

```sql
SELECT
    TO_CHAR(o.order_purchase_timestamp, 'Month') AS month,
    ROUND(SUM(p.payment_value)) AS total_sales
FROM amazon_brazil.orders o
JOIN amazon_brazil.payments p
    ON o.order_id = p.order_id
GROUP BY month
ORDER BY total_sales DESC
LIMIT 3;
```

Data Output Messages Notifications

SQL Show

	month text	total_sales double precision
1	May	5240703
2	August	5090465
3	July	4976771

Q.5	**Amazon India is interested in product categories with significant price variations.** Find categories where the difference between the maximum and minimum product prices is greater than 500 BRL. ○ **Output**: product_category_name, price_difference

Query | Query History

```
33  SELECT
34      p.product_category_name,
35      MAX(oi.price) - MIN(oi.price) AS price_difference
36  FROM amazon_brazil.order_items oi
37  JOIN amazon_brazil.product p
38      ON oi.product_id = p.product_id
39  GROUP BY p.product_category_name
40  HAVING MAX(oi.price) - MIN(oi.price) > 500;
```

Data Output | Messages | Notifications

SQL

	product_category_name character varying	price_difference double precision
1	agro_industria_e_comercio	2977.01
2	alimentos_bebidas	693.4
3	artes	6495.5
4	artigos_de_festas	563.21
5	audio	584.09
6	automotivo	2254.51
7	bebes	3895.46
8	bebidas	617
9	beleza_saude	3122.8
10	brinquedos	1695.09
11	cama_mesa_banho	1992.99
12	casa_conforto	792.01

Q.6	**To enhance the customer experience, Amazon India wants to find which payment types have the most consistent transaction amounts.** Identify the payment types with the least variance in transaction amounts, sorting by the smallest standard deviation first. ○ **Output**: payment_type, std_deviation

Query Query History

```
2  FROM amazon_brazil."payments"
3
4  SELECT
5      payment_type,
6      STDDEV(payment_value) AS std_deviation
7  FROM amazon_brazil.payments
8  GROUP BY payment_type
9  ORDER BY std_deviation ASC;
```

Data Output Messages Notifications

SQL

	payment_type character varying	std_deviation double precision
1	not_defined	0
2	voucher	115.51251707905554
3	boleto	213.57746283589506
4	credit_card	222.11834661265323
5	debit_card	245.73979872814786

Q.7	**Amazon India wants to identify products that may have incomplete name in order to fix it from their end**. Retrieve the list of products where the product category name is missing or contains only a single character. ○ **Output**: product_id, product_category_name

Query | Query History

```sql
SELECT *
    FROM amazon_brazil."product";

SELECT
    product_id,
    product_category_name
FROM amazon_brazil."product"
WHERE product_category_name IS NULL
   OR LENGTH(TRIM(product_category_name)) = 1;
```

Data Output | Messages | Notifications

SQL

	product_id character varying	product_category_name character varying
1	a41e356c76fab66334f36de622ecb...	[null]
2	d8dee61c2034d6d075997acef1870...	[null]
3	56139431d72cd51f19eb9f7dae4d1...	[null]
4	46b48281eb6d663ced748f324108c...	[null]
5	5fb61f482620cb672f5e586bb132e...	[null]
6	e10758160da97891c2fdcbc35f0f0...	[null]
7	39e3b9b12cd0bf8ee681bbc1c130f...	[null]
8	794de06c32a626a5692ff50e4985d...	[null]
9	7af3e2da474486a3519b0cba9dea8...	[null]
10	629beb8e7317703dcc5f35b5463fd...	[null]
11	3a78f64aac654298e4b9aff32fc218...	[null]

Analysis - II

Q.1	**Amazon India wants to understand which payment types are most popular across different order value segments (e.g., low, medium, high).** Segment order values into three ranges: orders less than 200 BRL, between 200 and 1000 BRL, and over 1000 BRL. Calculate the count of each payment type within these ranges and display the results in descending order of count ○ **Output:** order_value_segment, payment_type, count

Query | Query History

```sql
SELECT
    CASE
        WHEN payment_value < 200 THEN 'Low'
        WHEN payment_value BETWEEN 200 AND 1000 THEN 'Medium'
        ELSE 'High'
    END AS order_value_segment,
    payment_type,
    COUNT(*) AS count
FROM amazon_brazil."payments"
GROUP BY order_value_segment, payment_type
ORDER BY count DESC;
```

Data Output | Messages | Notifications

	order_value_segment text	payment_type character varying	count bigint
1	Low	credit_card	181644
2	Low	boleto	49332
3	Medium	credit_card	45909
4	Low	voucher	16428
5	Medium	boleto	9486
6	Low	debit_card	3861
7	High	credit_card	2832
8	Medium	voucher	858
9	Medium	debit_card	681
10	High	boleto	534

Q.2	**Amazon India wants to analyse the price range and average price for each product category.** Calculate the minimum, maximum, and average price for each category, and list them in descending order by the average price. ○ **Output**: `product_category_name, min_price, max_price, avg_price`

Query | Query History

```
42  SELECT
43      p.product_category_name,
44      MIN(oi.price) AS min_price,
45      MAX(oi.price) AS max_price,
46      AVG(oi.price) AS avg_price
47  FROM amazon_brazil.order_items oi
48  JOIN amazon_brazil.product p
49      ON oi.product_id = p.product_id
50  GROUP BY p.product_category_name
51  ORDER BY avg_price DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 79

	product_category_name character varying	min_price double precision	max_price double precision	avg_price double precision
1	pcs	34.5	6729	1098.3405418719203
2	portateis_casa_forno_e_cafe	10.19	2899	624.2856578947369
3	eletrodomesticos_2	13.9	2350	476.12495798319276
4	agro_industria_e_comercio	12.99	2990	341.6610426540283
5	instrumentos_musicais	4.9	4399.87	281.6159999999988
6	eletroportateis	6.5	4799	280.77846833578707
7	portateis_cozinha_e_preparadores_de_aliment...	17.42	1099	264.5686666666667
8	telefonia_fixa	6	1790	225.6931818181822
9	construcao_ferramentas_seguranca	8.9	3099.9	208.99237113402097
10	relogios_presentes	8.99	3999.9	200.91187708750272

Q.3	**Amazon India wants to identify the customers who have placed multiple orders over time.** Find all customers with more than one order, and display their customer unique IDs along with the total number of orders they have placed. ○ **Output:** `customer_unique_id, total_orders`

Query | Query History

```
11  SELECT
12      c.customer_unique_id,
13      COUNT(o.order_id) AS total_orders
14  FROM amazon_brazil.orders o
15  JOIN amazon_brazil.customer_data c
16      ON o.customer_id = c.customer_id
17  GROUP BY c.customer_unique_id
18  HAVING COUNT(o.order_id) > 1;
```

Data Output | Messages | Notifications

SQL

	customer_unique_id character varying	total_orders bigint
1	0000366f3b9a7992bf8c76cfdf3221e2	10
2	0000b849f77a49e4a4ce2b2a4ca5be...	10
3	0000f46a3911fa3c08054444833370...	10
4	0000f6ccb0745a6a4b88665a16c9f0...	10
5	0004aac84e0df4da2b147fca70cf8255	10
6	0004bd2a26a76fe21f786e4fbd80607f	10
7	00050ab1314c0e55a6ca13cf7181fecf	10
8	00053a61a98854899e70ed204dd4b...	10
9	0005e1862207bf6ccc02e4228effd9a0	10
10	0005ef4cd20d2893f0d9fbd94d3c0d...	10
11	0006fdc98a402fceb4eb0ee528f6a8d4	10
12	00090324bbad0e9342388303bb71b...	10

Q.4	**Amazon India wants to categorize customers into different types ('New – order qty. = 1' ; 'Returning' –order qty. 2 to 4; 'Loyal' – order qty. >4) based on their purchase history.** Use a temporary table to define these categories and join it with the customers table to update and display the customer types. ○ **Output**: customer_unique_id, customer_type

Query Query History

```
20  CREATE TEMP TABLE customer_orders AS
21  SELECT
22      c.customer_unique_id,
23      COUNT(o.order_id) AS total_orders
24  FROM amazon_brazil.orders o
25  JOIN amazon_brazil.customer_data c
26      ON o.customer_id = c.customer_id
27  GROUP BY c.customer_unique_id;
28
29  SELECT
30      customer_unique_id,
31      CASE
32          WHEN total_orders = 1 THEN 'New'
33          WHEN total_orders BETWEEN 2 AND 4 THEN 'Returning'
34          ELSE 'Loyal'
35      END AS customer_type
36  FROM customer_orders;
```

Data Output Messages Notifications

SQL Sho

	customer_unique_id character varying	customer_type text
1	0000366f3b9a7992bf8c76cfdf3221e2	Loyal
2	0000b849f77a49e4a4ce2b2a4ca5be...	Loyal
3	0000f46a3911fa3c08054444833370...	Loyal
4	0000f6ccb0745a6a4b88665a16c9f0...	Loyal
5	0004aac84e0df4da2b147fca70cf8255	Loyal

Q.5	**Amazon India wants to know which product categories generate the most revenue.** Use joins between the tables to calculate the total revenue for each product category. Display the top 5 categories. ○ **Output**: product_category_name, total_revenue

Query Query History

```
54
55  SELECT
56      p.product_category_name,
57      SUM(oi.price + oi.freight_value) AS total_revenue
58  FROM amazon_brazil.order_items oi
59  JOIN amazon_brazil.product p
60      ON oi.product_id = p.product_id
61  GROUP BY p.product_category_name
62  ORDER BY total_revenue DESC
63  LIMIT 5;
```

Data Output Messages Notifications

SQL

	product_category_name character varying	total_revenue double precision
1	beleza_saude	5761134.520000016
2	relogios_presentes	5214142.840000027
3	cama_mesa_banho	4944359.63999998
4	esporte_lazer	4616766.640000004
5	informatica_acessorios	4230613.120000005

Analysis - III

Q.1	**The marketing team wants to compare the total sales between different seasons.** Use a subquery to calculate total sales for each season (Spring, Summer, Autumn, Winter) based on order purchase dates, and display the results. Spring is in the months of March, April and May. Summer is from June to August and Autumn is between September and November and rest months are Winter. ○ **Output:** season, total_sales

Query | Query History

```
40  SELECT
41      season,
42      SUM(total_sales) AS total_sales
43  FROM (
44      SELECT
45          CASE
46              WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (3,4,5) THEN 'Spring'
47              WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (6,7,8) THEN 'Summer'
48              WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (9,10,11) THEN 'Autumn'
49              ELSE 'Winter'
50          END AS season,
51          p.payment_value AS total_sales
52      FROM amazon_brazil.orders o
53      JOIN amazon_brazil.payments p
54          ON o.order_id = p.order_id
55  ) sub
56  GROUP BY season;
```

Data Output | Messages | Notifications

Showing rows: 1 to 4 | Page No: 1

	season text	total_sales double precision
1	Winter	10248854.009999061
2	Autumn	8300085.179999651
3	Spring	14804970.600000085
4	Summer	14672706.569999335

Q.2	**The inventory team is interested in identifying products that have sales volumes above the overall average.** Write a query that uses a subquery to filter products with a total quantity sold above the average quantity. ○ **Output:** `product_id, total_quantity_sold`

Q.3	**To understand seasonal sales patterns, the finance team is analysing the monthly revenue trends over the past year (year 2018).** Run a query to calculate total revenue generated each month and identify periods of peak and low sales. Export the data to Excel and create a graph to visually represent revenue changes across the months. ○ **Output:** `month, total_revenue`

Query | Query History

```
60  SELECT
61      DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
62      SUM(p.payment_value) AS total_revenue
63  FROM amazon_brazil.orders o
64  JOIN amazon_brazil.payments p
65      ON o.order_id = p.order_id
66  WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
67  GROUP BY month
68  ORDER BY month;
```

Data Output | Messages | Notifications

SQL | Showing rows: 1

	month timestamp with time zone	total_revenue double precision
1	2018-01-01 00:00:00+05:30	3345012.5400000103
2	2018-02-01 00:00:00+05:30	2977390.0200000037
3	2018-03-01 00:00:00+05:30	3478956.359999981
4	2018-04-01 00:00:00+05:30	3482356.439999992
5	2018-05-01 00:00:00+05:30	3461946.4499999806
6	2018-06-01 00:00:00+05:30	3071641.499999988
7	2018-07-01 00:00:00+05:30	3199622.2500000014
8	2018-08-01 00:00:00+05:30	3067275.95999999
9	2018-09-01 00:00:00+05:30	13318.619999999999
10	2018-10-01 00:00:00+05:30	1769.01

Q.4	**A loyalty program is being designed for Amazon India.** Create a segmentation based on purchase frequency: 'Occasional' for customers with 1-2 orders, 'Regular' for 3-5 orders, and 'Loyal' for more than 5 orders. Use a CTE to classify customers and their count and generate a chart in Excel to show the proportion of each segment. ○ **Output:** `customer_type, count`

Query | Query History

```
71
72  WITH customer_counts AS (
73      SELECT
74          c.customer_unique_id,
75          COUNT(o.order_id) AS total_orders
76      FROM amazon_brazil.orders o
77      JOIN amazon_brazil.customer_data c
78          ON o.customer_id = c.customer_id
79      GROUP BY c.customer_unique_id
80  )
81  SELECT
82      CASE
83          WHEN total_orders BETWEEN 1 AND 2 THEN 'Occasional'
84          WHEN total_orders BETWEEN 3 AND 5 THEN 'Regular'
85          ELSE 'Loyal'
86      END AS customer_type,
87      COUNT(*) AS count
88  FROM customer_counts
89  GROUP BY customer_type;
```

Data Output | Messages | Notifications

	customer_type text	count bigint
1	Loyal	95077

Q.5	**Amazon wants to identify high-value customers to target for an exclusive rewards program.** You are required to rank customers based on their average order value (avg_order_value) to find the top 20 customers. ○ **Output:** `customer_id, avg_order_value, and customer_rank`

Query | Query History

```sql
SELECT
    customer_id,
    AVG(payment_value) AS avg_order_value,
    RANK() OVER (ORDER BY AVG(payment_value) DESC) AS customer_rank
FROM amazon_brazil.orders o
JOIN amazon_brazil.payments p
    ON o.order_id = p.order_id
GROUP BY customer_id
LIMIT 20;
```

Data Output | Messages | Notifications

SQL | Showing rows: 1

	customer_id character varying	avg_order_value double precision	customer_rank bigint
1	1617b1357756262bfa56ab541c47...	13664.08	1
2	ec5b2ba62e574342386871631fafd...	7274.88	2
3	c6e2731c5b391845f6800c97401a4...	6929.31	3
4	f48d464a0baaea338cb25f816991a...	6922.21	4
5	3fd6777bbce08a352fddd04e4a7cc...	6726.66	5
6	05455dfa7cd02f13d132aa7a6a972...	6081.54	6
7	df55c14d1476a9a3467f131269c24...	4950.34	7
8	e0a2412720e9ea4f26c1ac985f6a7...	4809.44	8
9	24bbf5fd2f2e1b359ee7de94defc4a...	4764.34	9
10	3d979689f636322c62418b6346b1...	4681.78	10
11	1afc82cd60e303ef09b4ef9837c950...	4513.32	11

Q.6	**Amazon wants to analyze sales growth trends for its key products over their lifecycle.** Calculate monthly cumulative sales for each product from the date of its first sale. Use a recursive CTE to compute the cumulative sales (total_sales) for each product month by month. ○ Output: `product_id, sale_month, and total_sales`

Q.7	**To understand how different payment methods affect monthly sales growth, Amazon wants to compute the total sales for each payment method and calculate the month-over-month growth rate for the past year (year 2018).** Write query to first calculate total monthly sales for each payment method, then compute the percentage change from the previous month. ○ Output: `payment_type, sale_month, monthly_total, monthly_change.`